

```
In [ ]: #Array Creation Functions using Numpy:
```

```
In [2]: import numpy as np
```

```
In [ ]: #Create an array from a list
```

```
In [3]: a = np.array([1,2,3])  
print("Array a:",a)
```

```
Array a: [1 2 3]
```

```
In [ ]: #Create an array with evenly spaced values
```

```
In [4]: b = np.arange(0,10,2)  
print("Array b:",b)
```

```
Array b: [0 2 4 6 8]
```

```
In [ ]: #Create an array filled with zeros
```

```
In [5]: d = np.zeros((2,3))  
print("Array d:\n",d)
```

```
Array d:  
[[0. 0. 0.]  
 [0. 0. 0.]]
```

```
In [6]: d = np.zeros((2,3), dtype = int)      #Convert to int  
print("Array d:\n",d)
```

```
Array d:  
[[0 0 0]  
 [0 0 0]]
```

```
In [ ]: #Create an array filled with ones
```

```
In [7]: e = np.ones((3,2))  
print("Array e:\n",e)
```

```
Array e:  
[[1. 1.]  
 [1. 1.]  
 [1. 1.]]
```

```
In [8]: e = np.ones((3,2), dtype = int)      #Convert to int  
        print("Array e:\n",e)
```

```
Array e:  
[[1 1]  
 [1 1]  
 [1 1]]
```

```
In [ ]: #Create an identity matrix
```

```
In [11]: f = np.eye(4)  
         print("Identity Matrix f:\n",f)
```

```
Identity Matrix f:  
[[1. 0. 0. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 1. 0.]  
 [0. 0. 0. 1.]]
```

```
In [12]: f = np.eye(4, dtype=int)          #Convert to int  
         print("Identity Matrix f:\n",f)
```

```
Identity Matrix f:  
[[1 0 0 0]  
 [0 1 0 0]  
 [0 0 1 0]  
 [0 0 0 1]]
```

```
In [ ]: #Array Manipulation Functions:
```

```
In [ ]: #Reshape an array:
```

```
In [19]: a1 = np.array([1,2,3])  
         reshaped = np.reshape(a1,(1,3))  
         print("Reshaped array a1:\n",a1)
```

Reshaped array a1:
[1 2 3]

```
In [28]: a2 = np.array([1,2,3,4,5,6,7,8])  
a2
```

```
Out[28]: array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
In [31]: reshaped2 = np.reshape(a2,(1,8))  
print("Reshaped array a2:\n",a2)
```

Reshaped array a2:
[1 2 3 4 5 6 7 8]

```
In [ ]: #Flatten an array:
```

```
In [32]: f1 = np.array([[1,2],[3,4]])  
f1
```

```
Out[32]: array([[1, 2],  
               [3, 4]])
```

```
In [33]: flattened = np.ravel(f1)  
print("Flattened array:",flattened)
```

Flattened array: [1 2 3 4]

```
In [34]: f2 = np.array([[1,2],[3,4],[5,6],[7,8]])  
f2
```

```
Out[34]: array([[1, 2],  
               [3, 4],  
               [5, 6],  
               [7, 8]])
```

```
In [35]: flattened2 = np.ravel(f2)  
print("Flattened array:",flattened2)
```

Flattened array: [1 2 3 4 5 6 7 8]

```
In [ ]: #Transpose an array:
```

```
In [36]: e1 = np.array([[1,2],[3,4]])  
e1
```

```
Out[36]: array([[1, 2],  
               [3, 4]])
```

```
In [37]: transposed = np.transpose(e1)  
print("Transposed array:\n",transposed)
```

```
Transposed array:  
[[1 3]  
 [2 4]]
```

```
In [39]: e2 = np.array([[1,2],[3,4],[5,6],[7,8]])  
e2
```

```
Out[39]: array([[1, 2],  
               [3, 4],  
               [5, 6],  
               [7, 8]])
```

```
In [40]: transposed2 = np.transpose(e2)  
print("Transposed array:\n",transposed2)
```

```
Transposed array:  
[[1 3 5 7]  
 [2 4 6 8]]
```

```
In [ ]: #Stack array vertically:
```

```
In [41]: a2 = np.array([1,2])  
b2 = np.array([3,4])  
stacked = np.vstack([a2,b2])  
print("Stacked array:\n",stacked)
```

```
Stacked array:  
[[1 2]  
 [3 4]]
```

```
In [ ]: #Stack array horizontally:
```

```
In [42]: a2 = np.array([1,2])  
b2 = np.array([3,4])  
stacked = np.hstack([a2,b2])  
print("Stacked array:\n",stacked)
```

Stacked array:
[1 2 3 4]

```
In [ ]:
```